**Project Title:** Deep Learning for Satellite-Based Wildfire Detection

**Project Category (in the form of Method + Application, e.g. CNN for Face Recognition):** Convolutional Neural Networks (CNNs) for Wildfire Detection

**Names with UVA IDs:** Isaac Tabor (zds3st), Michael Dunlap (upw4ys), Jarrett Markman (gzt8na)

**Repository Link:** https://github.com/isaactabor21/ds6050-final-project/

**Proposal:**

1. **Motivation: What problem are we tackling? Is this an application or a theoretical result?**

The goal of our project is to help address the problem of wildfire detection by building a deep learning model capable of identifying wildfire-affected regions from satellite imagery, by classifying satellite image patches as either wildfire or no wildfire. This is primarily an application result.

According to NASA, extreme wildfire activity has more than doubled worldwide, and has been amplified due to Earth's warming climate, particularly in northern and temperate forests, and satellite data is rich in helping detect and track them, by helping communities and land managers prepare for and respond to fires and understand this growing risk. NASA's Terra and Aqua satellites detect active wildfires twice each day. In 2023, Canada's warmest and driest conditions since 1980 stoked extreme fires that lasted for five months. NASA researchers found that these fires released about 640 million metric tons of carbon. Isaac, being from Minnesota, has also experienced many summer days in Minnesota with unhealthy air quality due to Canadian wildfires.

Training a CNN to classify image patches as "fire" or "no fire" could enhance early detection of wildfires, that could be timelier and more scalable for environmental management and disaster response than manually analyzing large volumes of satellite imagery.

2. **Dataset: URL + Description of the dataset with some basic stats**

We are using the Wildfire Prediction Dataset (Satellite Images) from Kaggle.

Kaggle URL: https://www.kaggle.com/datasets/abdelghaniaaba/wildfire-prediction-dataset

The creator generated these images by extracting satellite imagery using the Map Box API at latitude and longitude coordinates of wildfire events (>0.01 acres burned) from the Canadian government wildfire dataset. All images are colored. Original wildfire records come from the Government of Canada Open Data Portal URL: https://open.canada.ca/data/en/dataset/9d8f219c-4df0-4481-926f-8a2a532ca003

Description with some basic stats:

Dataset contains satellite images (350px x 350 px) in 2 classes: Wildfire (22710 images) and No wildfire (20140 images). File names are coordinates, so a stretch goal could be to aggregate predicted probabilities and map it. The data was divided into Train (~70%), Test (~15%), Validation (~15%).

**Load the data**

1. First, download the zip file from Kaggle

```
# CODE TO UNZIP THE FILE
zip_file_path = 'archive.zip'
extract_dir = '.' # '.' is current directory

# Try extracting with the `unzip` shell command
!unzip -q {zip_file_path} -d {extract_dir}
```

```
replace ./test/nowildfire/-113.91777,50.901087.jpg? [y]es, [n]o, [A]ll, [N]one, [r]ename: y
replace ./test/nowildfire/-113.91782,50.896917.jpg? [y]es, [n]o, [A]ll, [N]one, [r]ename: y
replace ./test/nowildfire/-113.91793,51.091205.jpg? [y]es, [n]o, [A]ll, [N]one, [r]ename: y
replace ./test/nowildfire/-113.918016,50.978447.jpg? [y]es, [n]o, [A]ll, [N]one, [r]ename: A
```

Dataset contains satellite images (350px x 350 px) in 2 classes: Wildfire (22710 images) and No wildfire (20140 images). File names are coordinates, so a stretch goal could be to aggregate predicted probabilities and map it. The data was divided into Train (70%), Test (15%), Validation (15%).

This means 3 folders:

- a test folder
- a train folder
- a valid folder

With each folder containing 2 folders:

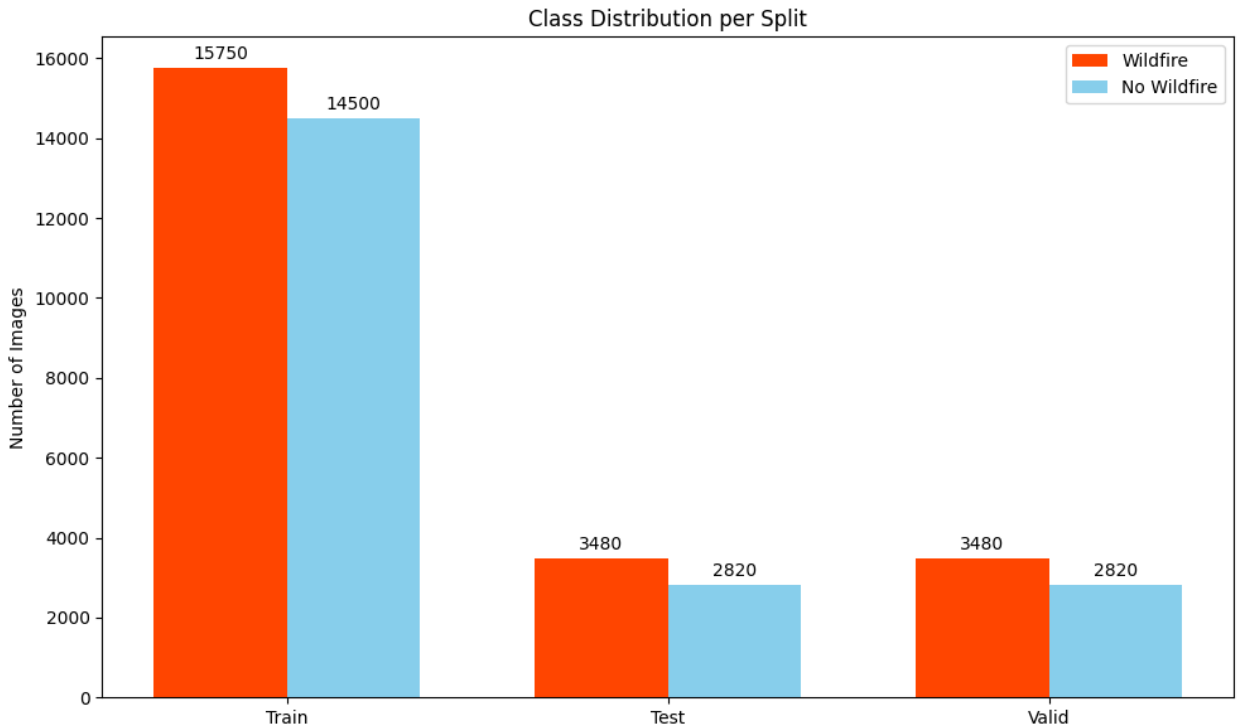
- `nowildfire` folder
- `wildfire` folder

Here after downloading the data from Kaggle, we've read in the zip file into a Python environment with data containing three different data folders (train/test/valid), with each folder containing a folder of nowildfire and wildfire with images.

Examples of a wildfire (left) and a nowildfire (right) image:

Interestingly, wild fire images did not have higher red intensity! This might mean the 'smoke' or 'haze' is more dominant than the flames.

Of the 70/15/15 train/test/validation split on all the images in the dataset with wildfire and nowildfire, we have balanced classes across each split.




To utilize other model architectures (such as ResNet and VGG) and to ensure that the model trains faster and takes up less space, we resized the 350x350 pixel images to 224x224 images.

```python
import torch
from torchvision import datasets, transforms
from torch.utils.data import DataLoader
from PIL import ImageFile
ImageFile.LOAD_TRUNCATED_IMAGES = True

# Basic transformations
transform = transforms.Compose([
    transforms.Resize((224, 224)),
    transforms.ToTensor(),
])

# Load the datasets using your folder structure
train_data = datasets.ImageFolder('train', transform=transform)
test_data = datasets.ImageFolder('test', transform=transform)
val_data = datasets.ImageFolder('valid', transform=transform)

# Dataloaders: These feed the model in "batches"
train_loader = DataLoader(train_data, batch_size=32, shuffle=True)
test_loader = DataLoader(test_data, batch_size=32, shuffle=False)
val_loader = DataLoader(val_data, batch_size=32, shuffle=False)

print(f"Classes found: {train_data.classes}") # Should be ['nowildfire', 'wildfire']
```

```
Classes found: ['nowildfire', 'wildfire']
```

### 3. Related Work: At least 2 examples of prior methodology on the topic is a valuable addition

Spiller, D., Carbone, A., Amici, S., Thangavel, K., Sabatini, R., & Laneve, G. (2023). Wildfire Detection Using Convolutional Neural Networks and PRISMA Hyperspectral Imagery: A Spatial-Spectral Analysis. Remote Sensing, 15(19), 4855. https://doi.org/10.3390/rs15194855

- This research paper examined the generalization capability of four neural network models: the fully connected (FC), one-dimensional (1D) CNN, two-dimensional (2D) CNN, and three-dimensional (3D) CNN model
- They used training data from Australia and Sicily and evaluated the performances of the trained model on test dataset from Oregon.
- Their results were that their FC architecture generalized best, while their 3D CNN model offered “more refined and less distorted classifications”

- However, they had a persistent issue with false fire detection and confusion between smoke and shadows

M. S. Khan, R. Patil and S. Ali Haider, "Application of Convolutional Neural Networks For Wild Fire Detection," 2020 SoutheastCon, Raleigh, NC, USA, 2020, pp. 1-5, doi: 10.1109/SoutheastCon44009.2020.9368283.

- These results were very promising as a CNN with one convolution layer, 64 classification nodes, and 2 fully connected layers gave the best accuracy

4. **Technical Plan: What are the inputs and outputs of our task? Which deep learning models and loss functions do we plan to use?**

The inputs to our tasks are satellite images split into test, train, and validation sets containing images labeled either 0 for no wildfire or 1 for wildfire. This is determined based on a threshold of 0.01 acres burned. The output for our task will therefore be a model that will predict within the test set either wildfire or no wildfire. In this sense, we hope to create a model that can look at an image and classify it into one of these two groups. The deep learning model we intend to use is a CNN, because of the one-to-one basis of our dataset and task. In our plan, we believe that false positives and false negatives should be weighed equally, with maybe a slight preference towards giving false positives (i.e. model predicting wildfire when it doesn't meet the threshold). Therefore, we intend to use BCEWithLogitsLoss as our loss function. This is designed for binary classification and includes a parameter that we can adjust to define which incorrect decision we prefer (PyTorch Documentation).

To start, we built a simple CNN model with one convolution layer, one pooling layer, and then one fully connected layer to show that we can build a basic model with the data. Our activation function here is ReLu.

```python
import torch.nn as nn
import torch.nn.functional as F

class SimpleWildfireCNN(nn.Module):
    def __init__(self):
        super().__init__()
        self.conv1 = nn.Conv2d(3, 16, 3, padding=1)
        self.pool = nn.MaxPool2d(2, 2)
        self.fc1 = nn.Linear(16 * 112 * 112, 2) # Downsampled from 224 to 112

    def forward(self, x):
        x = self.pool(F.relu(self.conv1(x)))
        x = torch.flatten(x, 1)
        x = self.fc1(x)
        return x

model = SimpleWildfireCNN()
```

Utilizing our basic model, we were able to make a prediction (albeit a false positive, as the image is classified as wildfire despite not actually being wildfire).

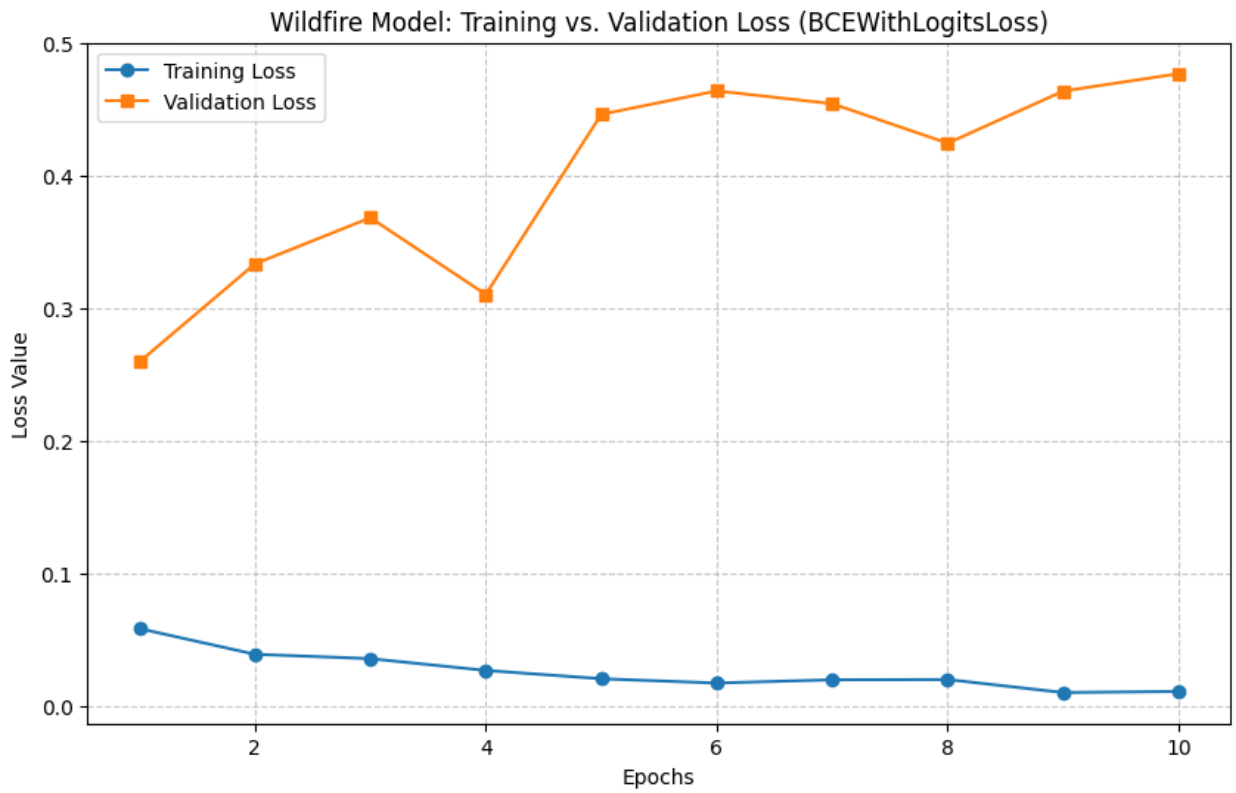
```python
# 1. Grab one image from your test set
images, labels = next(iter(test_loader))
single_img = images[0].unsqueeze(0) # Get the first image in the batch

# 2. Pass it through the model
model.eval() # Set to evaluation mode
with torch.no_grad():
    output = model(single_img)
    prediction = torch.argmax(output, dim=1).item()

# 3. Show results
class_names = train_data.classes
print(f"Model thinks this image is: {class_names[prediction]}")
print(f"The actual label is: {class_names[labels[0]]}")
```

```
Model thinks this image is: wildfire
The actual label is: nowildfire
```

Using Binary Cross-Entropy with logits loss as our loss function, we can plot how our trained and validation data sets loss converges over multiple epochs.




As we can see, the training loss converges quickly; however, the validation loss increases, indicating that our model likely will not be effective in predicting new data. To combat this, we can likely add in more layers (potentially using an existing model architecture), as well as implementing batch normalization, pooling, and dropout while training our model so that our validation loss decreases ensuring an effective model.

5. **Evaluation Plan: What experiments are we planning to run? How do we plan to evaluate our ML algorithm?**

Our main experiment is to measure how well we can predict wildfires or not, utilizing train/validation/test splits to maximize metrics such as accuracy. Utilizing our results, we plan to visualize model predictions across Canada to identify what areas are more at risk, and how these wildfires are coming in based on the geography and ultimately use that to help prevent and protect the country and areas from wildfires. Lastly, we plan to compare and expand our CNN model, architecture, and results based on prior works, such as Spiller et al. (2023) and Khan et al. (2020), to serve as a benchmark for our model's performance.

Something additional to be aware of is the cost of the predictions we are making and how they would be applied. A false positive (yes wildfire despite nowildfire) can be harmful because of the potential harms to the area by unnecessarily expending resources as well as frightening those in the area and putting them at risk. On the other hand, a false negative (nowildfire despite yes wildfire) is arguably even more costly, failing to act upon the need of the people, potentially leading damages to infrastructure, wildlife, and people, by failing to prepare in advance. To better prepare around this, we plan to incorporate elements of sensitivity (true positive rate) and specificity (true negative rate), along with overall accuracy with the goal of building a) an overall accurate and effective model and b) a sensitive and specific model, so that with our model results we can measure how wildfires present themselves across the Canadian landscape and visualize where they present themselves.

6. **References**

- https://science.nasa.gov/earth/explore/wildfires-and-climate-change/
- https://docs.pytorch.org/docs/stable/generated/torch.nn.BCEWithLogitsLoss.html